```
now.utc                # => Mon, 07 Jun 2010 23:27:52 +0000
```
This method is also aliased as `getutc`.

16.1.2.3 `utc?`

The predicate `utc?` says whether the receiver has UTC as its time zone:
```
now = DateTime.now # => Mon, 07 Jun 2010 19:30:47 -0400
now.utc?           # => false
now.utc.utc?       # => true
```
16.1.2.4 `advance`

The most generic way to jump to another datetime is `advance`. This method receives a hash with keys `:years`, `:months`, `:weeks`, `:days`, `:hours`, `:minutes`, and `:seconds`, and returns a datetime advanced as much as the present keys indicate.
```
d = DateTime.current
# => Thu, 05 Aug 2010 11:33:31 +0000
d.advance(years: 1, months: 1, days: 1, hours: 1, minutes: 1, seconds:
1)
# => Tue, 06 Sep 2011 12:34:32 +0000
```
This method first computes the destination date passing `:years`, `:months`, `:weeks`, and `:days` to `Date#advance` documented above. After that, it adjusts the time calling `since` with the number of seconds to advance. This order is relevant, a different ordering would give different datetimes in some edge-cases. The example in `Date#advance` applies, and we can extend it to show order relevance related to the time bits.

If we first move the date bits (that have also a relative order of processing, as documented before), and then the time bits we get for example the following computation:

```
d = DateTime.new(2010, 2, 28, 23, 59, 59)
# => Sun, 28 Feb 2010 23:59:59 +0000
d.advance(months: 1, seconds: 1)
# => Mon, 29 Mar 2010 00:00:00 +0000
```
but if we computed them the other way around, the result would be different:

```
d.advance(seconds: 1).advance(months: 1)
# => Thu, 01 Apr 2010 00:00:00 +0000
```
Since `DateTime` is not DST-aware you can end up in a non-existing point in time with no warning or error telling you so.

16.1.3 Changing Components

The method `change` allows you to get a new datetime which is the same as the receiver except for the given options, which may include `:year`, `:month`, `:day`, `:hour`, `:min`, `:sec`, `:offset`, `:start`:
```
now = DateTime.current
# => Tue, 08 Jun 2010 01:56:22 +0000
now.change(year: 2011, offset: Rational(-6, 24))
# => Wed, 08 Jun 2011 01:56:22 -0600
```
If hours are zeroed, then minutes and seconds are too (unless they have given values):

```
now.change(hour: 0)
# => Tue, 08 Jun 2010 00:00:00 +0000
```
Similarly, if minutes are zeroed, then seconds are too (unless it has given a value):

```
now.change(min: 0)
# => Tue, 08 Jun 2010 01:00:00 +0000
```
This method is not tolerant to non-existing dates, if the change is invalid `ArgumentError` is raised:
```
DateTime.current.change(month: 2, day: 30)
# => ArgumentError: invalid date
```
16.1.4 Durations